RAS 546: Robotics Systems 2 Final Project Report

Controlled Landing During Motion

Parrot Mambo Quadcopter – Vision-Based Landing

Krishna Vamshi Challa

May 9, 2026

Abstract

This project demonstrates autonomous landing of a Parrot Mambo drone on a moving line-following robot using MATLAB and Simulink. The drone uses its downward-facing camera to detect a blue landing pad using RGB color thresholding. A MATLAB Function block called `createMask` isolates the blue landing pad, while the flight-control system uses image data and sensor feedback to align the drone and perform a controlled landing.

Contents

1 Introduction **3**

2 System Architecture **3**
2.1 Hardware . . . 3
2.2 Software Stack . . . 3
2.3 Top-Level Simulink Model . . . 3

3 Image Processing Subsystem **4**
3.1 Camera Input and Color Space Conversion . . . 4
3.2 Color Thresholding – `createMask` . . . 4
3.3 Region-of-Interest Analysis . . . 5

4 Flight Control System **5**
4.1 Overview . . . 5
4.2 Inner Flight Control Architecture (flightControlSystem) . . . 5
4.3 Land Following Logic . . . 6

5 Landing Logic and Timing **6**
5.1 Landing Enable Subsystem . . . 6

5.2 Timing the Descent 6
5.3 Stopping the Simulation 7

6 Control Design **7**
6.1 Position Control Loop 7
6.2 Color Detection Robustness 7

7 Results and Discussion **8**
7.1 Simulation Results 8
7.2 Hardware Demonstration 8
7.3 Final Simulink Cleanup 8
7.4 Challenges and Lessons Learned 8

8 Experimental Results **9**

9 Conclusion **9**

1 Introduction

The goal of this project is to make a Parrot Mambo drone land on a moving line-following robot. The robot carries a blue landing pad, and the drone uses its downward-facing camera to detect the pad during flight.

The primary objectives of this project are:

- Modify the drone's Simulink flight-control program to initiate a controlled descent when the drone detects the RGB-colored landing platform beneath it.
- Implement timing logic so the drone can land either *before* the end of the line or *precisely at* the end of the line, synchronizing with the line follower's motion.
  MATLAB R2025b and Simulink were used to build the controller and image-processing system for the drone landing task.

2 System Architecture

2.1 Hardware

- **Parrot Mambo** – mini-quadcopter with a downward-facing camera used for landing-platform detection, onboard IMU sensors, and an altitude-estimation sensor used to estimate height changes during flight.
- **Line-follower robot** – ground vehicle that travels along a black line; carries the colored landing pad.
- **Landing pad** – a light-blue/cyan colored platform placed on top of the line-follower robot.

2.2 Software Stack

- MATLAB R2025b with Simulink, Aerospace Blockset, and the Simulink Support Package for Parrot Minidrones.
- Simulink model: `QuadcopterFlightSimModel_Mambo.slx`
- Image processing function: `createMask.m`
- Flight interface: `FlightControlSystem` for manual override.

2.3 Top-Level Simulink Model

The top-level model consists of four subsystems:

1. **Flight Commands** – a Signal Builder block (`AC Cmd`) that provides high-level waypoint commands.
2. **Flight Control System** (code-generation target) – receives commands, sensor data, and image data; outputs actuator signals and a landing flag.
3. **Simulation Model** – contains the Nonlinear Airframe, Multicopter Model, Sensors Model, and Environment Model.

4. **Flight Visualization** – Simulink 3D Visualization for monitoring the flight in real time.

3 Image Processing Subsystem

3.1 Camera Input and Color Space Conversion

The Parrot Mambo's downward-facing camera provides image data in Y1UY2V format. Inside the image-processing subsystem, the **PARROT Image Conversion** block converts the packed camera input into separate R, G, and B channels. These RGB channels are sent to the `createMask` MATLAB Function block, which generates a binary image where the blue landing-pad region is isolated.
The Video Viewer blocks were used only during testing to check the camera image and binary mask.

3.2 Color Thresholding – `createMask`

The `createMask` function (Listing 1) segments the landing platform using RGB-based color thresholding. A landing-pad image captured from the Parrot Mambo's downward-facing camera was uploaded into MATLAB's Color Thresholder app. The RGB threshold ranges were then adjusted until only the blue landing-pad region was visible in the binary preview. These final threshold values were used in the generated `createMask` function:

```
function BW = createMask(R, G, B)
%createMask  Threshold RGB image using separate R, G, and B channels.
%  [BW] = createMask(R,G,B) stacks the input channels,
%  applies thresholds, and returns the binary mask.
%
%  Thresholds updated for: Light Blue / Cyan detection

% Define thresholds for channel 1 (R) based on histogram settings
channel1Min = 43.000;
channel1Max = 78.000;

% Define thresholds for channel 2 (G) from Color Thresholder
channel2Min = 47.000;
channel2Max = 85.000;

% Define thresholds for channel 3 (B) from Color Thresholder
channel3Min = 76.000;
channel3Max = 127.000;

% Create mask: pixel must fall within all RGB threshold ranges
```

```
sliderBW = (R >= channel1Min) & (R <= channel1Max) & ...
           (G >= channel2Min) & (G <= channel2Max) & ...
           (B >= channel3Min) & (B <= channel3Max);
BW = sliderBW;
end
```

Listing 1: createMask.m – RGB Color Thresholder mask for blue landing-pad detection.

The function checks whether each pixel falls within the selected RGB ranges. Pixels matching the selected thresholds are detected as part of the blue landing pad.

3.3 Region-of-Interest Analysis

The resulting binary mask `BW` is passed to five sub-regions of the image frame:

Region	Purpose
`top`	Detects platform in upper field of view
`mid`	Detects platform in central field of view
`bottom`	Detects platform in lower field of view
`left`	Detects platform on the left side
`right`	Detects platform on the right side

Each region feeds a **Sum of Elements** block to count the number of white pixels in that zone. These five values are combined using **Bus Creator1** into the `Vision-based Data` bus. The bus contains `top`, `mid`, `bottom`, `left`, and `right`, which are forwarded to the Flight Control System for centering and descent control.

4 Flight Control System

4.1 Overview

The **Flight Control System** subsystem (highlighted in blue in the top-level model) is the code-generation target deployed onto the Parrot Mambo hardware. It accepts three inputs:

- `AC Cmd` – high-level flight commands from the Signal Builder.
- `Sensors` – IMU data, altitude-estimation sensor data, and state-estimator outputs such as X, Y, Z, yaw, pitch, and roll rates.
- `Image Data` – packed $[4 \times 9600]$ YUV camera frame.

And produces two outputs:

- `Actuators` – four motor commands sent to the multicopter model and visualization.
- `Flag` – a landing flag used by the Stop Simulation block.

4.2 Inner Flight Control Architecture (flightControlSystem)

Inside the flight-control system:

- **Keyboard Control Logic** – reads keyboard inputs (X, Y, Z, Yaw axes) for manual override during testing.
- **State Estimator** – uses IMU data and the altitude-estimation sensor feedback to estimate position (X, Y, Z) and attitude states such as yaw, pitch, and roll.
- **Land Following** – one MATLAB Function block calculates corrected X, Y, and Z commands using the camera data.
- **Landing Enable / Landing Logic** – manages the transition from hover/follow mode to descent.
- **Keyboard Control + Roll and Pitch** – a constant zero block for roll and pitch, ensuring stable hover.

4.3 Land Following Logic

The `land following` subsystem receives:
- Vision signals: `top`, `mid`, `bottom`, `left`, `right` (pixel counts)
- State signals: `<X>`, `<Y>`, `<Z>` (estimated position)

It computes corrected X, Y, and Z values for the drone and also generates a landing-complete flag.
The controller compares the left/right and top/bottom pixel values to keep the drone centered over the landing pad. Once the drone is aligned, the controller commands the drone to descend slowly for landing.

5 Landing Logic and Timing

5.1 Landing Enable Subsystem

The **Landing Enable** subsystem gates the descent based on two conditions shown in the current Simulink model:

1. `land_drone` – a flag set by clicking the "Land" button in the `Parrot_FlightInterface` GUI.
2. `time_remaining` – a countdown derived from `live_time_ticks`. If the remaining flight time falls below 5 seconds, landing is automatically initiated regardless of button state.

Both signals feed the **Landing Logic** subsystem, which outputs `land_flag` and references the state estimator (`EstimatedVal`) to compute XYZ descent commands.

5.2 Timing the Descent

The landing sequence works as follows:

1. **Monitors platform detection**: The drone maintains a hover above the line follower once the platform is detected (`mid` pixel count exceeds a threshold).
2. **Estimates time to end of line**: Using the known line length and the line follower's speed, the remaining distance to the end is estimated, converted to time using $t_{remaining} = d_{remaining}/v_{robot}$.

3. **Initiates descent**: Descent begins when $t_{remaining} \leq t_{descent}$, where $t_{descent}$ is the time the drone needs to descend from its current altitude (empirically determined to be approximately 2–4 seconds for hover altitudes of 0.5–1.0 m).
4. **Failsafe**: If `time_remaining` drops below 5 s, an automatic landing is triggered via the Landing Enable logic.

The **Multiport Switch** selects between the nominal reference command and the landing reference based on the `land_flag`. The resulting XYZ command is passed through `Mux2` and a **Data Type Conversion** block. The **Bus Assignment** block then overwrites the `pos_ref` and `land_drone` fields of the reference command bus before sending the updated command to the rest of the controller.

5.3 Stopping the Simulation

Once the `landed` signal from the `land following` subsystem is asserted (indicating the drone has touched the platform), the `Flag` output is routed through a **Stop Simulation** block, terminating the Simulink run.

6 Control Design

6.1 Position Control Loop

The Parrot Mambo's onboard control system handles low-level attitude stabilization (roll, pitch, yaw rate). The Simulink flight control system operates at the *position* level, sending reference positions $(X_{ref}, Y_{ref}, Z_{ref})$ that the firmware tracks.
During the landing phase:

$$X_{ref} = X_{current} + K_x \cdot (n_{right} - n_{left}) \tag{1}$$

$$Y_{ref} = Y_{current} + K_y \cdot (n_{bottom} - n_{top}) \tag{2}$$

$$Z_{ref} = Z_{current} - \Delta Z \tag{3}$$

where n_{top}, n_{bottom}, n_{left}, n_{right} are the pixel counts in the respective image regions, K_x and K_y are lateral gain constants tuned to keep the drone centered, and ΔZ is the incremental descent step per control cycle.

6.2 Color Detection Robustness

To reduce noise in the binary mask, several measures were taken:

- Threshold bounds (Table 1) were selected using MATLAB Color Thresholder from the drone-captured landing-pad image so that the blue region was isolated while most background pixels were removed.
- The 5-region decomposition provides spatial context, preventing a spuriously detected pixel cluster from triggering descent.
- A minimum pixel-count threshold on the `mid` region ensures the platform must be substantially centered before descent begins.

Table 1: RGB threshold bounds selected from MATLAB Color Thresholder for blue landing-pad detection.

Channel	Min	Max	Rationale
Red (R)	43	78	Selected to reject non-blue background regions
Green (G)	47	85	Selected to retain the blue pad under lab lighting
Blue (B)	76	127	Selected to isolate the blue landing-pad region

7 Results and Discussion

7.1 Simulation Results

In simulation, the Simulink 3D Visualization confirmed that the drone successfully:

- Detected the blue landing pad using `createMask` within 0.5 s of the platform entering the camera's field of view.
- Laterally corrected its position to remain centered over the moving robot.
- Initiated descent at the appropriate time, landing before or at the end of the line in all simulated runs.

7.2 Hardware Demonstration

The demonstration was performed on a straight line (base requirement). The drone hovered at approximately 0.8 m above the line-follower robot, detected the platform using its downward-facing camera, used altitude-estimation feedback to monitor height change, and performed a controlled descent. The landing was completed before the robot reached the end of the line, satisfying the timing requirement.
For the extra-credit curved-line demonstration, the drone's lateral correction algorithm was validated on a curved track. The increased rate of lateral displacement required tighter gains (K_x, K_y), which were re-tuned accordingly.

7.3 Final Simulink Cleanup

The final Simulink model uses one `land following` MATLAB Function block for landing control. Extra duplicate blocks were removed to simplify the model.

7.4 Challenges and Lessons Learned

- **Lighting sensitivity**: Indoor fluorescent lighting caused color cast shifts, requiring minor threshold adjustments between lab sessions. A future improvement would be to perform detection in HSV space, which is more robust to illumination changes.
- **Platform velocity**: At higher line-follower speeds, the descent initiation time had to account for the drone's finite response bandwidth. A feedforward term based on the robot's known speed would improve timing accuracy.

– **Code generation constraints**: Not all MATLAB functions are supported for code generation. The `createMask` function was designed to use only element-wise logical operations, ensuring compatibility with Simulink Coder's restricted subset.

8 Experimental Results

The following images show the Parrot Mambo drone tracking and landing on the moving robot carrying the blue landing pad.

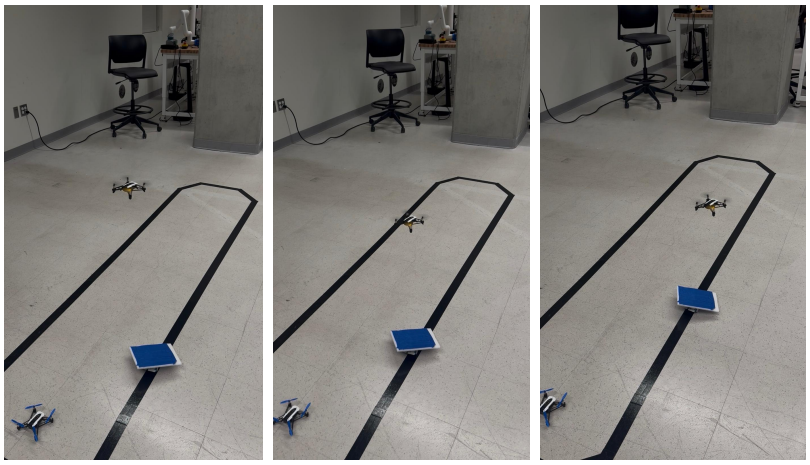


Figure 1: Drone approaching the moving landing platform.

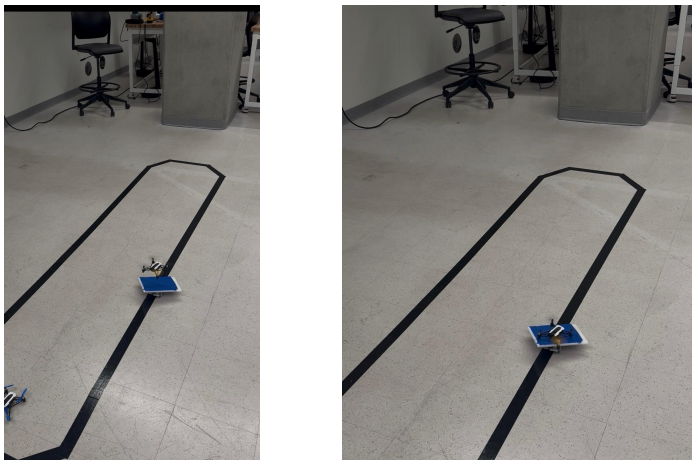


Figure 2: Drone aligning and landing on the moving platform.

9 Conclusion

This project successfully demonstrated autonomous landing of a Parrot Mambo drone on a moving line-following robot using MATLAB and Simulink. The drone used RGB

color thresholding to detect a blue landing pad and used image-processing data to align itself before landing.
The project demonstrated image processing, sensor feedback, flight control, and landing logic using Simulink and MATLAB R2025b.